

P2592R0

Hashing support for `std::chrono` value classes

Published Proposal, 2022-05-20

Author:

[Giuseppe D'Angelo](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

We propose to add `std::hash` specializations for the value classes in the `std::chrono` namespace.

Table of Contents

- 1 Changelog
- 2 Motivation and Scope
- 3 Impact On The Standard
- 4 Design Decisions
 - 4.1 What about `time_zone`?
- 5 Technical Specifications
 - 5.1 Feature testing macro
 - 5.2 Proposed wording

6 Acknowledgements

References

Informative References

§ 1. Changelog

- R0
 - First submission

§ 2. Motivation and Scope

The time library defines several value classes and class templates in the `std::chrono` namespace. All of the following are copyable and comparable for equality (and some are also default constructible, therefore satisfying `regular`):

- `duration`
- `time_point`
- `day`
- `month`
- `year`
- `weekday`
- `weekday_indexed`
- `weekday_last`
- `month_day`
- `month_day_last`
- `month_weekday`
- `month_weekday_last`
- `year_month`
- `year_month_day`
- `year_month_day_last`

- `year_month_weekday`
- `year_month_weekday_last`
- `zoned_time`
- `leap_second`

Unfortunately, **none** of these classes comes with an enabled `std::hash` specialization. This means that for instance it's not possible to store them in the Standard Library unordered associative containers:

```
std::unordered_set<std::chrono::milliseconds> unique_measurements; // ERROR
```

Of course users can work around this limitation by providing a custom hash, for instance something like this:

```
struct duration_hash
{
    template <class Rep, class Period>
    constexpr auto operator()(const std::chrono::duration<Rep, Period> &d)
        noexcept(/* ... */)
    {
        std::hash<Rep> h;
        return h(d.count());
    }
};

std::unordered_set<std::chrono::milliseconds, duration_hash> unique_measurements;
```

This limitation seems however to be unnecessarily vexing. Most of the types listed above can actually have a straightforward implementation for their hashing function, and there's little justification for the Standard Library itself not to provide it.

The only case where a slightly more challenging implementation may be required is `zoned_time`, because it may need to mix the hash for the time zone pointer with the hash for the time point. This is not however different from "prior art"; for instance, the implementation of `hash<error_code>` has to solve the same problem.

§ 3. Impact On The Standard

This proposal is a pure library addition.

This proposal does not depend on any other library extensions.

This proposal does not require any changes in the core language.

§ 4. Design Decisions

§ 4.1. What about `time_zone`?

While `time_zone` is comparable for equality, it is not a regular type because it is not copyable. Users that need to store `time_zone` objects typically use *pointers* to them. As such, we do not see the need to add a `hash<time_zone>` specialization, and we are not proposing it here.

The same reasoning applies to `time_zone_link`.

§ 5. Technical Specifications

All the proposed changes are relative to [\[N4910\]](#).

§ 5.1. Feature testing macro

In `[version.syn]`, modify

```
#define __cpp_lib_chrono 201907L-YYYYMML // also in <chrono>
```

by replacing the existing value with the year and month of adoption of the present proposal.

§ 5.2. Proposed wording

Add the following at the end of [time.syn]:

```
namespace std {
    // ???, hash support
    template<class T> struct hash;
    template<class Rep, class Period> struct hash<chrono::duration<Rep, Per
    template<class Clock, class Duration> struct hash<chrono::time_point<Cl
    template<> struct hash<chrono::day>;
    template<> struct hash<chrono::month>;
    template<> struct hash<chrono::year>;
    template<> struct hash<chrono::weekday>;
    template<> struct hash<chrono::weekday_indexed>;
    template<> struct hash<chrono::weekday_last>;
    template<> struct hash<chrono::month_day>;
    template<> struct hash<chrono::month_day_last>;
    template<> struct hash<chrono::month_weekday>;
    template<> struct hash<chrono::month_weekday_last>;
    template<> struct hash<chrono::year_month>;
    template<> struct hash<chrono::year_month_day>;
    template<> struct hash<chrono::year_month_day_last>;
    template<> struct hash<chrono::year_month_weekday>;
    template<> struct hash<chrono::year_month_weekday_last>;
    template<class Duration, class TimeZonePtr> struct hash<chrono::zoned_t
    template<> struct hash<chrono::leap_second>;
}
```

Add a new subclause after [time.parse], with the following content:

??? Hash support [time.hash]

```
template<class Rep, class Period> struct hash<chrono::duration<Rep, Period>
```

(1) Letting D be `chrono::duration<Rep, Period>`, the specialization `hash<D>` is enabled ([unord.hash]) if and only if `hash<Rep>` is enabled. When enabled, for an object d of type D, `hash<D>()(d)` evaluates to the same value as `hash<Rep>()(d.count())`. The member functions are not guaranteed to be `noexcept`.

```
template<class Clock, class Duration> struct hash<chrono::time_point<Clock, Duration>
```

(2) Letting TP be `chrono::time_point<Clock, Duration>`, the specialization `hash<TP>` is enabled ([unord.hash]) if and only if `hash<Duration>` is enabled. When enabled, for an object tp of type TP, `hash<TP>()(tp)` evaluates to the same value as `hash<Duration>()(tp.time_since_epoch())`. The member functions are not guaranteed to be `noexcept`.

```
template<> struct hash<chrono::day>;
template<> struct hash<chrono::month>;
template<> struct hash<chrono::year>;
template<> struct hash<chrono::weekday>;
template<> struct hash<chrono::weekday_indexed>;
template<> struct hash<chrono::weekday_last>;
template<> struct hash<chrono::month_day>;
template<> struct hash<chrono::month_day_last>;
template<> struct hash<chrono::month_weekday>;
template<> struct hash<chrono::month_weekday_last>;
template<> struct hash<chrono::year_month>;
template<> struct hash<chrono::year_month_day>;
template<> struct hash<chrono::year_month_day_last>;
template<> struct hash<chrono::year_month_weekday>;
template<> struct hash<chrono::year_month_weekday_last>;
```

(3) The specialization is enabled ([unord.hash]).

```
template<class Duration, class TimeZonePtr> struct hash<chrono::zoned_time<Duration, TimeZonePtr>
```

(4) Letting ZT be `chrono::zoned_time<Duration, TimeZonePtr>`, the specialization `hash<ZT>` is enabled (`[unord.hash]`) if and only if `hash<Duration>` is enabled and `hash<TimeZonePtr>` is enabled. The member functions are not guaranteed to be `noexcept`.

```
template<> struct hash<chrono::leap_second>;
```

(5) The specialization is enabled (`[unord.hash]`).

§ 6. Acknowledgements

Thanks to KDAB for supporting this work.

Thanks to Howard Hinnant for designing `<chrono>` and for discussing why hash support had not been added in the past.

All remaining errors are ours and ours only.

§ References

§ Informative References

[N4910]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4910.pdf>